

Databricks screenshots

This screenshot shows the first two code blocks of a Databricks notebook titled 'risk_scoring_improved'. The interface includes a sidebar with navigation options like Home, Workspace, Recents, and Jobs & Pipelines. The top bar shows the search bar and 'CTRL + P' shortcut.

Code Block 2: Imports necessary libraries for machine learning and data manipulation.

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

Code Block 3: Loads the training and score data from CSV files and prints their shapes.

```
# ---- load ----
df2 = pd.read_csv("./train.csv")
df3 = pd.read_csv("./score.csv")

print("train:", df2.shape)
print("score:", df3.shape)
```

Below the code, the shapes of the dataframes are displayed:

```
> df2: pandas.core.frame.DataFrame = [policy_id object, insured_id object ... 29 more fields]
> df3: pandas.core.frame.DataFrame = [policy_id object, insured_id object ... 26 more fields]
train: (15875, 31)
score: (3888, 28)
```

This screenshot shows the remaining code blocks of the Databricks notebook. It continues with data preprocessing and model evaluation.

Code Block 4: Preprocessing step.

```
# ---- preprocessing ----
```

Code Block 5: Evaluates the model and prints accuracy, ROC-AUC, and PR-AUC scores.

```
# ---- evaluate ----
from sklearn.metrics import roc_auc_score, accuracy_score, average_precision_score

preds = tmp.predict(X_test)
probs = tmp.predict_proba(X_test)[:, 1]

print("accuracy:", accuracy_score(y_test, preds))
print("ROC-AUC:", roc_auc_score(y_test, probs))
print("PR-AUC:", average_precision_score(y_test, probs))

# The prototype fits on X_train, then calls .predict() and reports accuracy on X_train again - it never touches X_test.
# and the next point is that the accuracy is a bad metrics here regardless. only 12.4% of policies have a claim,
# so a model that predicts "no claim" for every policy would have 87.6% accuracy. - its useless
# After removing leakage variables, the model performance dropped from perfect accuracy to a more realistic validation performance.
# This confirmed that the prototype model was relying on post-claim information.
```

The evaluation results are shown below the code:

```
accuracy: 0.8736483788536644
ROC-AUC: 0.6840593310345832
PR-AUC: 0.2383327744819795
```

Code Block 6: Scores new policies.

```
# ---- score new policies ----
X2 = df3[feat_cols]
```

The screenshot displays the Databricks workspace interface. On the left is a sidebar with navigation options: Home, Learn, Workspace (selected), Recent, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. Below these are sections for Data Engineering (Runs, Data Ingestion, Visual Data Prep), AI/ML (Playground, Agents, AI Gateway, Experiments), and a top navigation bar with 'Verify identity', 'workspace', and 'Share' buttons.

The main area shows a Python notebook titled 'risk_scoring_improved'. The code defines a function 'predict_proba' that uses a trained model to predict risk scores based on policy IDs. It then applies this function to a DataFrame of policy IDs, saves the results to a CSV file named 'predictions.csv', and prints the first few rows. Below the code, the output shows the schema and summary statistics of the resulting DataFrame.

```
# The original prototype used predict(), which returned only binary outcomes (0/1).
# To support policy ranking by risk, predict_proba() was introduced, generating a probability-based risk score instead of a binary prediction.

risk_scores = tmp.predict_proba(X2)[0, 1]

predictions = pd.DataFrame({
    "policy_id": df["policy_id"],
    "risk_score": risk_scores
})

predictions.to_csv("predictions.csv", index=False)

print("done -- predictions.csv written")
print(predictions.head())
print(predictions["risk_score"].describe())

> predictions: pandas.core.frame.DataFrame = [policy_id: object, risk_score: float64]
> X2: pandas.core.frame.DataFrame = [coverage: type: int32, vehicle_count: int64 ... 16 more fields]

done -- predictions.csv written
policy_id  risk_score
0  POL-0150001  0.203727
1  POL-0150002  0.212177
2  POL-0150003  0.153395
3  POL-0150004  0.149696
4  POL-0150005  0.178689

count    3000.000000
mean      0.129258
std       0.207479
min       0.021318
25%       0.075646
50%       0.116627
75%       0.163091
max       0.660867

Name: risk_score, dtype: float64
```